Security Vulnerability Assessment & Remediation Report

Multi-Lab Management System - Backend

Report Date: March 25, 2026 **System:** Node.js/Express Backend API **Scope:** Controllers, Middlewares, Routes, Server Configuration

---

Table of Contents

1. Executive Summary
2. Authentication Controller Security Fixes
3. Middleware Security Fixes
4. New Security Middlewares Created
5. Route Security Fixes
6. Application & Server Configuration
7. Dependencies Added
8. Summary & Statistics

Executive Summary

This report documents the comprehensive security hardening performed on the Multi-Lab Management System backend. The assessment identified and remediated vulnerabilities across multiple categories including authentication bypass, injection attacks, denial of service, and information disclosure.

Key Findings & Fixes

Category	Vulnerabilities Found	Status
Input Validation	15+	✅ Fixed
Authentication & Authorization	8	✅ Fixed
Rate Limiting	Not implemented	✅ Implemented
Injection Prevention	12+	✅ Fixed
Security Headers	Missing	✅ Added
Error Handling	Insecure	✅ Fixed

Authentication Controller Security Fixes

File: `src/controllers/auth.controller.js`

1. Timing Attack Prevention

Vulnerability: OTP and token comparisons used direct string comparison (`===`), allowing timing attacks to guess valid codes.

Risk Level: HIGH

Fix Applied:

```
const timingSafeEqual = (a, b) => {
  if (!a || !b || typeof a !== 'string' || typeof b !== 'string') return false;
  if (a.length !== b.length) return false;
```

```
  return crypto.timingSafeEqual(Buffer.from(a), Buffer.from(b));
};
```

Functions Fixed:

- `verifyStudentEmail()` - Email verification token
- `studentResetPassword()` - Password reset OTP
- `facultyResetPassword()` - Password reset OTP
- `staffResetPassword()` - Password reset OTP
- `staffChangePassword()` - Password change OTP
- `staffConfirmEmailChange()` - Email change OTP

---

2. Weak Password Policy

Vulnerability: No password strength requirements, allowing weak passwords like "123456".

Risk Level: HIGH

Fix Applied:

```
const PASSWORD_MIN_LENGTH = 8;
const isStrongPassword = (password) => {
  if (!password || password.length < PASSWORD_MIN_LENGTH) return false;
  const hasUppercase = /[A-Z]/.test(password);
  const hasLowercase = /[a-z]/.test(password);
  const hasNumber = /[0-9]/.test(password);
  return hasUppercase && hasLowercase && hasNumber;
};
```

Functions Fixed:

- `registerStudent()`
- `setPassword()` (Faculty)
- `studentResetPassword()`
- `facultyResetPassword()`
- `staffResetPassword()`
- `staffChangePassword()`

---

3. Email Format Validation

Vulnerability: No email format validation, allowing malformed emails and potential injection.

Risk Level: MEDIUM

Fix Applied:

```
const EMAIL_REGEX = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
const isValidEmail = (email) => {
  if (!email || typeof email !== 'string') return false;
  return EMAIL_REGEX.test(email) && email.length <= 254;
};
```

Functions Fixed: All functions accepting email input (12 functions)

---

4. Name Validation (XSS Prevention)

Vulnerability: User names accepted without validation, allowing script injection.

Risk Level: MEDIUM

Fix Applied:

```
const NAME_REGEX = /^[a-zA-Z\s.'-]{2,100}$/;
const isValidName = (name) => {
  if (!name || typeof name !== 'string') return false;
  return NAME_REGEX.test(name);
};
```

---

5. XSS in Email Templates

Vulnerability: User-provided data (names, emails) inserted directly into HTML email templates.

Risk Level: HIGH

Fix Applied:

```
const escapeHtml = (str) => {
  if (!str) return '';
  return String(str)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
};
```

Email Templates Fixed:

- Verification emails
- Password reset emails
- OTP emails

---

6. Weak Bcrypt Rounds

Vulnerability: Bcrypt rounds set to 10, below current security recommendations.

Risk Level: LOW

Fix Applied:

```
const BCRYPT_ROUNDS = 12;
const hashedPassword = await bcrypt.hash(password, BCRYPT_ROUNDS);
```

---

7. ReDoS Prevention

Vulnerability: User input used directly in regex patterns, allowing Regular Expression Denial of Service.

Risk Level: MEDIUM

Fix Applied:

```
const escapeRegex = (str) => {
  return str.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
};
```

---

8. Token Format Validation

Vulnerability: Verification tokens not validated for format before database lookup.

Risk Level: LOW

Fix Applied:

```
const HEX_TOKEN_REGEX = /^[a-fA-F0-9]{64}$/;
const OTP_REGEX = /^[0-9]{6}$/;
```

Middleware Security Fixes

1. Auth Middleware (`auth.middleware.js`)

Vulnerabilities Fixed:

Issue	Fix
No token length validation	Added max length check (500 chars)
No role whitelist	Added `VALID_ROLES` array validation
Verbose JWT errors	Generic error messages only
Full user object in request	Use `.lean()` for efficiency
No ObjectId validation	Validate user ID format

Code Added:

```
const VALID_ROLES = ['student', 'faculty', 'super_admin', 'incharge', 'assistant', 'admin'];

const isValidObjectId = (id) => {
  if (!id || typeof id !== 'string') return false;
  return mongoose.Types.ObjectId.isValid(id);
};
```

---

2. Bill Upload Middleware (`billUpload.middleware.js`)

Vulnerabilities Fixed:

Issue	Fix
No PDF magic bytes validation	Check for `%PDF` header
Dangerous filename characters	Sanitize filenames
No content inspection	Scan for malicious patterns
Path traversal	Remove `..` from paths

Code Added:

```
const PDF_MAGIC_BYTES = Buffer.from([0x25, 0x50, 0x44, 0x46]); // %PDF

const sanitizeFilename = (filename) => {
  return filename
    .replace(/[^a-zA-Z0-9._-]/g, '_')
    .replace(/\.{2,}/g, '.')
    .slice(0, 100);
};

const validatePDFContent = (req, res, next) => {
  // Check magic bytes
  // Scan for JavaScript/embedded content
};
```

---

3. Role Middleware (`role.middleware.js`)

Vulnerabilities Fixed:

Issue	Fix
No role whitelist validation	Validate against allowed roles
Single role only	Support multiple roles
No initialization validation	Check config at startup

New Security Middlewares Created

1. Rate Limiter Middleware (`rateLimiter.middleware.js`)

Purpose: Prevent brute force and denial of service attacks.

Rate Limiters Created:

Limiter	Window	Max Requests	Use Case
`loginLimiter`	15 min	5	Login endpoints
`passwordResetLimiter`	15 min	3	Password reset
`registrationLimiter`	1 hour	5	User registration
`otpLimiter`	15 min	5	OTP verification
`emailChangeLimiter`	1 hour	3	Email change
`apiLimiter`	15 min	300	General API
`uploadLimiter`	1 hour	20	File uploads
`searchLimiter`	1 min	30	Search endpoints

---

2. Parameter Validator Middleware (`paramValidator.middleware.js`)

Purpose: Validate and sanitize route parameters and query strings.

Validators Created:

Validator	Purpose
`validateObjectId()`	MongoDB ObjectId format validation
`validateAssetTag()`	Asset tag format (alphanumeric + hyphen/underscore)
`sanitizeSearch()`	Escape regex special characters
`validatePaginationParams`	Limit page/limit values
`validateDateRange()`	Valid date format and logical order
`validateStatus()`	Whitelist allowed status values
`validateLabId()`	Lab ID format validation

Protection Against:

- NoSQL Injection via malformed ObjectIds
- ReDoS via search queries
- DoS via unlimited pagination
- Invalid date range queries

Route Security Fixes

Auth Routes

Files Modified:

- `auth/student.routes.js`
- `auth/staff.routes.js`
- `auth/faculty.routes.js`

Security Applied:

Endpoint	Rate Limiter
POST /login	`loginLimiter` (5/15min)
POST /register	`registrationLimiter` (5/hour)
POST /forgot-password	`passwordResetLimiter` (3/15min)
POST /reset-password	`passwordResetLimiter` (3/15min)
GET /verify-email	`otpLimiter` (5/15min)
POST /change-password	`otpLimiter` (5/15min)
POST /request-email-change	`emailChangeLimiter` (3/hour)
POST /confirm-email-change	`otpLimiter` (5/15min)

Admin Routes

Files Modified:

- `admin/index.js`
- `admin/bills.routes.js`

- `admin/componentRequest.routes.js`
- `admin/damagedAssets.routes.js`
- `admin/items.routes.js`
- `admin/labTransactions.routes.js`
- `admin/overdue.routes.js`
- `admin/profile.routes.js`
- `admin/transactions.routes.js`

Security Applied:

Route Parameter	Validation
`:id`	ObjectId format
`:labId`	ObjectId format
`:assetTag`	Alphanumeric format
`?q`, `?search`	Regex escape
`?page`, `?limit`	Numeric bounds (1-1000, 1-100)
`?startDate`, `?endDate`	Valid date format
`?status`	Whitelist validation

Assistant Routes

Files Modified:

- `assistant/index.js`
- `assistant/labSession.routes.js`
- `assistant/profile.routes.js`
- `assistant/transactions.routes.js`

Security Applied:

- Auth/role middleware at index level
- ObjectId validation on `:transaction_id`, `:itemId`
- Search sanitization
- Pagination validation
- OTP rate limiting on profile changes

Faculty Routes

Files Modified:

- `faculty/index.js`
- `faculty/approval.routes.js`
- `faculty/dashboard.routes.js`

Security Applied:

- API rate limiting on public token routes
- ObjectId validation on `:transaction_id`
- Pagination validation

Student Routes

Files Modified:

- `student/index.js`
- `student/componentRequest.routes.js`
- `student/items.routes.js`
- `student/profile.routes.js`
- `student/transactions.routes.js`

Security Applied:

- Auth/role middleware at index level
- ObjectId validation on `:item_id`, `:transaction_id`
- Search sanitization
- Pagination validation
- OTP rate limiting on password change

Super Admin Routes

Files Modified:

- `super_admin/index.js`
- `super_admin/analytics.routes.js`
- `super_admin/bills.routes.js`
- `super_admin/componentRequests.routes.js`
- `super_admin/damagedAssets.routes.js`
- `super_admin/inventory.routes.js`
- `super_admin/labs.routes.js`
- `super_admin/labSession.routes.js`
- `super_admin/labTransfer.routes.js`
- `super_admin/staff.routes.js`
- `super_admin/transactions.routes.js`

Security Applied:

- Auth/role middleware at index level
- LabId validation on all lab-specific routes
- ObjectId validation on all `:id`, `:staffId` params
- Date range validation on analytics
- Search sanitization
- Pagination validation

Application & Server Configuration

File: `src/app.js`

Security Headers (Helmet)

Added:

```
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      scriptSrc: ["'self'"],
      imgSrc: ["'self'", "data:", "https:"],
    },
  },
```

```
  crossOriginEmbedderPolicy: false,
}));
```

Headers Set:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `X-XSS-Protection: 1; mode=block`
- `Strict-Transport-Security`
- `Content-Security-Policy`

Body Size Limits

Vulnerability: No request body size limit allowed DoS via large payloads.

Fix:

```
app.use(express.json({ limit: '10kb' }));
app.use(express.urlencoded({ extended: true, limit: '10kb' }));
```

NoSQL Injection Prevention

Custom Sanitizer:

```
const sanitizeObject = (obj) => {
  for (const key in obj) {
    if (key.startsWith('$') || key.includes('.')) {
      delete obj[key];
    }
  }
};
```

HTTP Parameter Pollution Prevention

Added:

```
app.use(hpp({
  whitelist: ['sort', 'fields', 'page', 'limit']
}));
```

CORS Hardening

Before: Static origin array **After:** Dynamic origin validation with development flexibility

```
origin: (origin, callback) => {
  if (!origin && process.env.NODE_ENV !== 'production') {
    return callback(null, true);
  }
  if (allowedOrigins.includes(origin)) {
    return callback(null, true);
  }
  return callback(new Error('Not allowed by CORS'));
}
```

Error Handler (Information Disclosure Prevention)

Added:

```
app.use((err, req, res, next) => {
  const message = process.env.NODE_ENV === 'production'
    ? 'Internal server error'
    : err.message;
  res.status(err.status || 500).json({ error: message });
});
```

Server Fingerprint Removal

Added:

```
app.disable('x-powered-by');
```

Dependencies Added

Package	Version	Purpose
`helmet`	^8.x	Security headers
`hpp`	^0.2.x	HTTP Parameter Pollution prevention
`express-rate-limit`	^7.x	Rate limiting

Removed (Incompatible):

- `express-mongo-sanitize` - Replaced with custom sanitizer

Summary & Statistics

Files Modified

Category	Count
Controllers	1
Middlewares	4
New Middlewares	2
Route Files	30
Configuration	1
Total	**38**

Vulnerabilities Fixed by Category

Category	Count	Severity
Timing Attacks	6	HIGH

XSS (Cross-Site Scripting)	5	HIGH
NoSQL Injection	8	HIGH
ReDoS (Regex DoS)	4	MEDIUM
Brute Force	8 endpoints	HIGH
Weak Passwords	6	HIGH
Input Validation	15+	MEDIUM
Information Disclosure	3	MEDIUM
Missing Security Headers	6	MEDIUM
DoS (Body Size)	1	MEDIUM
HTTP Parameter Pollution	1	LOW

Security Measures Implemented

Measure	Status
Rate Limiting	✅ 8 different limiters
Input Validation	✅ All user inputs
Output Encoding	✅ HTML escaping
Authentication Hardening	✅ Timing-safe comparisons
Password Policy	✅ Strength requirements
Security Headers	✅ Helmet configured
CORS	✅ Strict origin validation
Error Handling	✅ No stack trace leakage
Parameter Validation	✅ All route params
File Upload Security	✅ PDF validation

Recommendations for Future

1. **Add CAPTCHA** - For registration and password reset forms
2. **Implement Account Lockout** - After N failed login attempts
3. **Add Request Signing** - For sensitive API endpoints
4. **Enable HTTPS Only** - Redirect all HTTP to HTTPS
5. **Add Security Monitoring** - Log and alert on suspicious activity
6. **Regular Dependency Audits** - Run `npm audit` weekly
7. **Penetration Testing** - Schedule quarterly security assessments

---

Conclusion

The Multi-Lab Management System backend has been comprehensively hardened against common web application vulnerabilities. All critical and high-severity issues have been addressed, with proper input validation, rate limiting, and security headers now in place.

Report Prepared By: Security Assessment Team **Date:** March 25, 2026

This document should be reviewed and updated after each security assessment or significant code changes.